

PDS-FPGAs

PRÁCTICA P4

FILTROS COMPENSADORES DE LAS RESPUESTAS EN FRECUENCIA DEL CIC Y DAC



UNIVERSITAT
POLITÈCNICA
DE VALENCIA

Índice

1	Objetivos	1
2	P4_1: Implementación del filtro compensador de la respuesta del CIC	1
2.1	Ficheros	3
2.2	Tareas a realizar	4
3	P4_2: Implementación del filtro compensador del DAC	6
3.1	Ficheros	8
3.2	Tareas a realizar	8

1. Objetivos

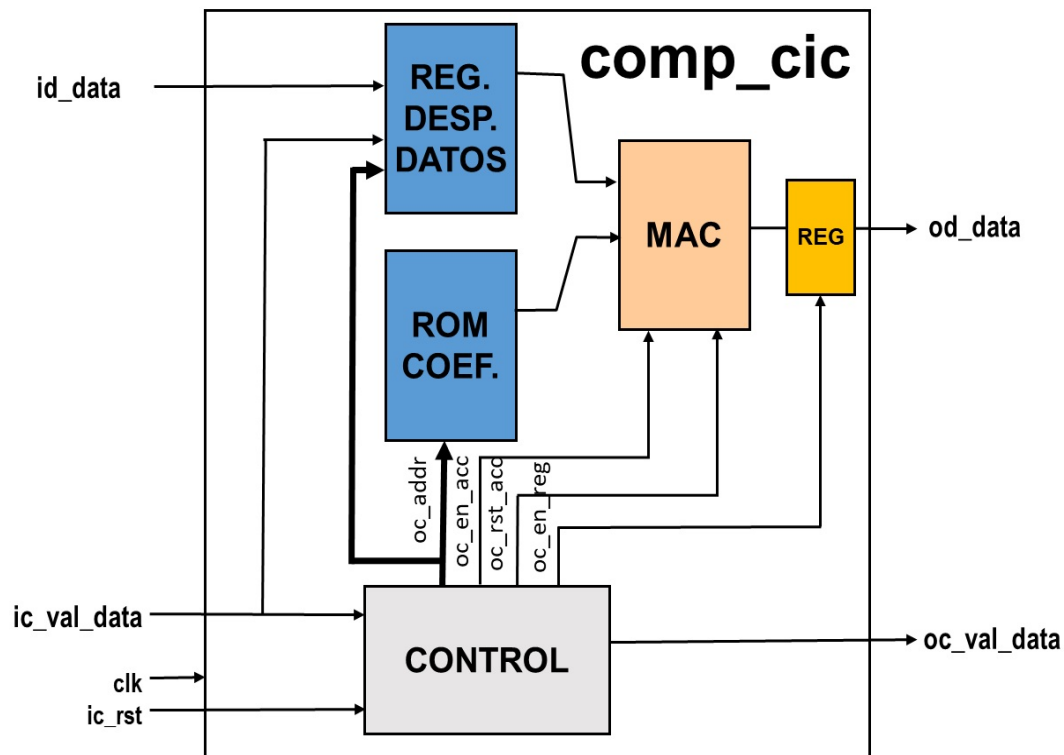
Los objetivos de esta práctica son:

- 1. Modelar con el lenguaje System Verilog un filtro secuencial que compense la respuesta del filtro interpolador CIC realizado en la práctica P3. El modelo deberá ser sintetizable y podrá ser implementado en un dispositivo FPGA Cyclone IV funcionando a una frecuencia de reloj de al menos 125 MHz.
- 2. Modelar con el lenguaje System Verilog un filtro paralelo que compense la respuesta del convertidor digital analógico. El diseño del filtro se realizará utilizando multiplicadores cableados (codificación CSD). El modelo deberá ser sintetizable y podrá ser implementado en un dispositivo FPGA Cyclone IV funcionando a una frecuencia de reloj de al menos 125 MHz.

2. P4_1: Implementación del filtro compensador de la respuesta del CIC

Se desea implementar un filtro FIR secuencial que compensará la respuesta del CIC. La frecuencia de muestreo de los datos de entrada será de 48 kHz, mientras que la frecuencia de reloj será de 96 MHz.

A continuación, se muestra el esquema del módulo *comp_cic*, que implementa el filtro compensador de la respuesta del interpolador CIC:



El módulo *comp_cic* dispone de los siguientes puertos:

- ***id_data***: entrada de datos a filtrar con formato S[16,15]
- ***ic_val_data***: entrada (binaria) que indica la presencia de una muestra válida en ***id_data***
- ***ic_rst***: entrada de reset síncrono activo a nivel alto. Este reset únicamente se utilizará en el módulo de control.
- ***clk***: entrada de reloj
- ***od_data***: salida de datos filtrados con formato S[18, 15]
- ***oc_val_data***: salida (binaria) que indica la presencia de una salida válida por ***od_data***

Las especificaciones del filtro compensador son:

- Número de coeficientes del filtro: Num_coef = 17
- Los coeficientes se proporcionan en el script de matlab `comp_cic_model.m`
- Ancho de banda de la señal de entrada: Bw = 20 kHz

Antes de comenzar a codificar todos los módulos, se deberá estudiar el modelo Simulink `comp_cic_model.slx` para determinar el crecimiento de la ruta de datos del filtro. Se aconseja implementar en primer lugar el filtro con precisión completa, y, una

vez se halle correctamente testado, recortar la salida del mismo para obtener una cuantificación S[18, 15].

La entrada del circuito `comp_cic_pc` recibe datos válidos a una frecuencia de muestreo de 48 kHz.

2.1. Ficheros

Ficheros necesarios para realizar la práctica P4_1:

- `*comp_cic_reg_mux.sv`: módulo que codifica un registro de desplazamiento direccionable
- `*comp_cic_reg_mux_tsb.sv`: banco de pruebas del módulo `comp_cic_reg_mux`
- `*comp_cic_mult_acc.sv`: módulo que codifica un multiplicador con acumulador de los productos
- `*comp_cic_mult_acc_tsb.sv`: banco de pruebas del módulo `comp_cic_mult_acc`
- `*comp_cic_control.sv`: módulo que codifica la máquina de estados necesaria para controlar el funcionamiento del filtro secuencial
- `*comp_cic_control_tsb.sv`: banco de pruebas del módulo `comp_cic_control`
- `*comp_cic_rom.sv`: módulo que codifica la memoria que almacena los coeficientes del filtro
- `*rom_coefs_comp_cic.txt`: fichero que contiene los coeficientes del filtro codificados en binario. Se deberá generar desde el script de Matlab
- `*comp_cic_rom_tsb.sv`: banco de pruebas del módulo `comp_cic_rom`
- `*comp_cic_pc.sv`: módulo que codifica el filtro con precisión completa
- `comp_cic.sv`: módulo que codifica el filtro con salida cuantificada con formato S[18, 15]
- `*comp_cic_tsb.sv`: banco de pruebas de los módulos `comp_cic_pc` y `comp_cic`
- `*comp_cic_tsb_pkg.sv`: Paquete para configurar el banco de pruebas de los módulos `comp_cic_pc` y `comp_cic`. Se generará desde Matlab
- `*comp_cic_tsb.do`: Configuración de las formas de onda del banco de pruebas del módulo `comp_cic_tsb` en el visor “waves” de Modelsim
- `*comp_cic_tsb.tcl`: Fichero para ejecutar la simulación en modo comandos, desde la consola de Modelsim.
- `comp_cic_model.slx`: Modelo de Simulink del modulador

- `sim_comp_cic_model.m`: Script de Matlab para configurar el modelo de Simulink `comp_cic_model.slx`
- `comp_cic_wrp.sv`: fichero “wrap” del módulo `comp_cic` para la medida de la frecuencia máxima de funcionamiento y para la simulación a nivel de puertas
- `*comp_cic_wrp_tsb.sv`: banco de pruebas del módulo `comp_cic_wrp` para realizar la simulación a nivel de puertas
- `comp_cic.sdc`: Fichero que contiene las restricciones temporales, para realizar el estudio de la velocidad del circuito

NOTA: Los ficheros marcados con un “*” no están disponibles en PoliformaT y los tendrá que generar el alumno.

2.2. Tareas a realizar

1. Revise el modelo Simulink suministrado (`comp_cic_model.slx`)
2. Utilice el fichero matlab de simulación (`sim_comp_cic_model.m`) y el modelo Simulink del filtro (`comp_cic_model.slx`) para realizar un análisis de:
 - La cuantificación que tendrá que aplicar a los coeficientes (`S[Wcoef Fcoef]`) de manera que aproveche los multiplicadores embebidos
 - El crecimiento de la ruta de datos (`Ng`) para obtener la salida con precisión completa cuantificada con formato `S[Wout Fout]`

Se proporciona el modelo sin cuantificar, duplíquelo para implementar el modelo cuantificado con la finalidad de comparar ambas respuestas. Recuerde que deberá cuantificar los coeficientes (`S[Wcoef Fcoef]`) en ambos modelos para asegurarse de que está simulando la misma respuesta frecuencial del filtro.

Simule el funcionamiento comparando las salidas del bloque de referencia con el bloque cuantificado. Testee el sistema con tres casos:

- Caso 1: Una señal cuadrada de frecuencia 5 kHz
 - Caso 2: Una señal senoidal de frecuencia 10 kHz
 - Caso 3: La señal impulso unitario discreto para simular la respuesta al impulso del filtro
3. Codifique, usando System Verilog, todos los módulos que conforman el filtro compensador de la respuesta del CIC. Estos módulos serán:
 - `comp_cic_reg_mux.sv`: Incluye el registro de desplazamiento y el multiplexor de salida. Se parametriza la cuantificación del dato de entrada (`Win`), y el número de coeficientes del filtro (`Num_coef`). Posee dos entradas de control, `ic_en_reg`, que habilita la carga de un nuevo dato, e `ic_sel`, que será el bus que direcciona el multiplexor. Realizar el banco de pruebas de este módulo para comprobar el comportamiento del mismo.

- **comp_cic_mult_acc.sv**: Incluye el multiplicador y el acumulador de salida. Se parametriza la cuantificación del dato de entrada (Win), la cuantificación de los coeficientes (Wcoef) y el crecimiento de la cuantificación (Ng). Posee dos entradas de control, **ic_rst_ac** para resetear la realimentación del acumulador, e **ic_en_ac** que habilita la acumulación. Realizar el banco de pruebas de este módulo para comprobar el comportamiento del mismo.
- **comp_cic_rom.sv**: Memoria ROM que contiene los coeficientes del filtro. Se parametriza la cuantificación de los coeficientes (Wcoef) y el número de coeficientes (Num_coef).
- **comp_cic_control.sv**: Máquina de estados que implementa el control de todos los módulos anteriores para realizar el filtro con una arquitectura hardware secuencial. Antes de codificar esta máquina, se deberá dibujar un diagrama de estados, que incluya toda la información necesaria para su implementación (señales que dan lugar a cambios de estado, salidas en cada estado etc). Se deberá parametrizar el número de coeficientes del filtro (Num_coef). Este módulo posee los siguientes puertos:
 - **ic_val_data**: señal utilizada para habilitar la entrada de una nueva muestra y comenzar el filtrado de la misma
 - **ic_rst**: reset asíncrono, activo a nivel alto
 - **oc_addr**: señal de control que se utilizarán para direccionar el multiplexor del módulo *comp_cic_reg_mux* y la salida de la memoria *comp_cic_rom*
 - **oc_rst_acc**: señal de control que se utilizará para resetear la realimentación del acumulador del módulo *comp_cic_mult_acc*
 - **oc_en_acc**: señal de control que se utilizarán para habilitar la acumulación en el acumulador del módulo *comp_cic_mult_acc*
 - **oc_en_reg**: señal de control que se utilizarán para habilitar el registro de salida del filtro

Realizar el banco de pruebas para comprobar el comportamiento de la máquina de estados.

4. Codifique, utilizando los módulos anteriores, el filtro compensador del CIC con precisión completa **comp_cic_pc.sv**. Los parámetros de este módulo deberán ser la cuantificación del dato de entrada (Win), la cuantificación de los coeficientes (Wcoef), el crecimiento de la cuantificación (Ng) y el número de coeficientes del filtro (Num_coef).
5. Realice un banco de pruebas para verificar el módulo *comp_cic_pc* comparándolo con su modelo de Simulink, y compruebe su correcto funcionamiento para los tres casos expuestos. Utilice para ello el fichero **comp_cic_model.slx**, que contiene el modelo Simulink con precisión finita, y se configura con el script de Matlab **sim_comp_cic_model.m**. En el banco de pruebas los valores de todas las entradas del módulo se deben introducir desde ficheros generados desde el script de configuración de Matlab y las salidas se deben comparar con las generadas por el modelo de Simulink, igual que se hizo en las prácticas anteriores.

Complete el script de Matlab `sim_comp_cic_model.m`, incluyendo el código necesario para generar los ficheros de test (`id_comp_cic.txt` y `od_comp_cic.txt`), así como el paquete con los parámetros necesarios para configurar el banco de pruebas `comp_cic_tsb_pkg.sv` y el fichero con los coeficientes del filtro codificados en complemento a dos, `rom_coefs_comp_cic.txt`.

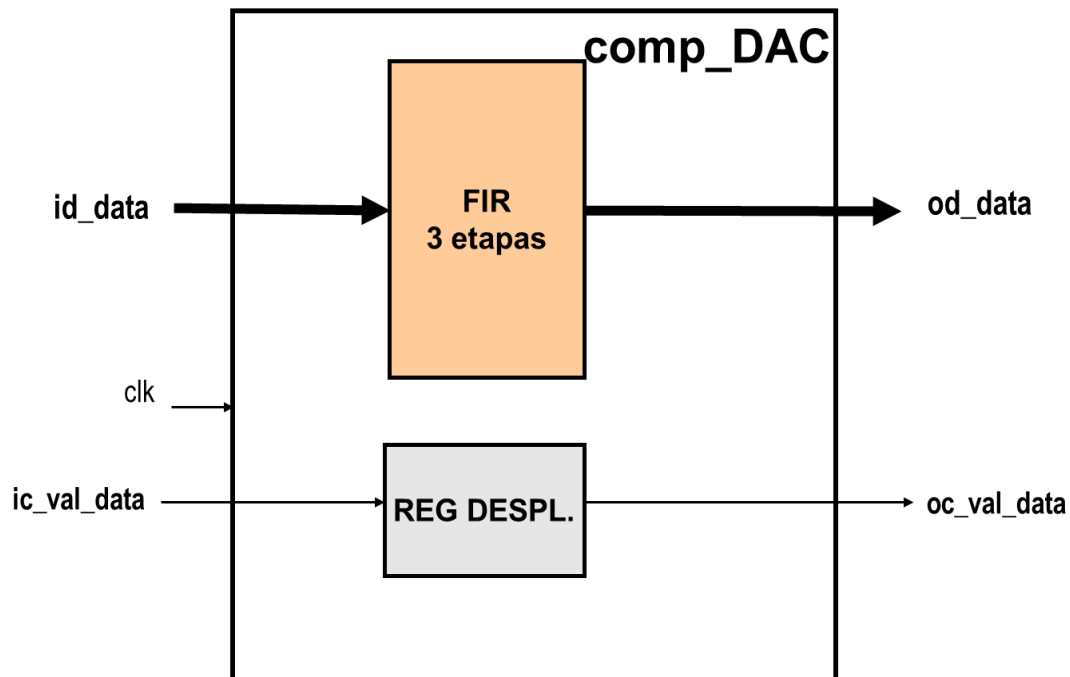
Una vez conseguido el funcionamiento correcto del filtro compensador de la respuesta del CIC con precisión completa, pasaremos a implementar el filtro con precisión recortada a 18 bits, de manera que la señal **od_data** se cuantifique en formato S[18,15]. Para ello, realice los siguientes pasos:

6. Utilice el modelo Simulink y el Script de Matlab para cuantificar la señal de salida del filtro con formato S[18,15].
7. Codifique el módulo `comp_cic` (`comp_cic.sv`), a partir del módulo `comp_cic_pc` con la finalidad de obtener como formato de salida S[18,15].
8. Realice un banco de pruebas (`comp_cic_tsb.sv`) para verificar el funcionamiento del módulo `comp_cic`, siguiendo la misma estrategia de verificación realizada anteriormente. Compruebe que el número de errores obtenido es igual a cero.
9. Una vez haya finalizado la depuración del módulo, escriba el script para automatizar la simulación del módulo con la herramienta Modelsim, `comp_cic_tsb.tcl`.
10. Estime el número de recursos hardware (Multiplicadores embebidos, LE aritméticos y FFs) que se requieren para la implementación del módulo `comp_cic`. Implemente el módulo `comp_cic` en el dispositivo Cyclone IV EP4CE115F29C7 e indique los recursos hardware que se requiere para su implementación (introduzca pantallazo de la ventana recursos hardware). Razone si los recursos obtenidos son coherentes con su estimación.
11. Escriba el código System Verilog del fichero `comp_cic_wrp.sv`, con un "wrap" para el módulo `comp_cic`, en el que se registren sus entradas y salidas, y obtenga la frecuencia máxima de funcionamiento en el dispositivo indicado.
12. Realice la **simulación a nivel de puertas** desde Quartus para verificar que el sistema, una vez sintetizado, no presenta problemas de temporización.

3. P4_2: Implementación del filtro compensador del DAC

Se desea modelar, con el lenguaje System Verilog, un filtro FIR paralelo que compensará la respuesta en frecuencia del convertidor digital analógico (DAC). La tasa de muestreo de la señal de entrada será la misma que la de reloj, y deberá llegar a 125 MHz.

Se requiere una implementación que utilice la codificación Canonic Signed Digit, explicada en el tema de Circuitos Aritméticos I para la implementación de multiplicadores por coeficientes constantes.



El módulo *comp_dac* presenta los siguientes puertos:

- ***id_data***: entrada del dato, cuantificado con formato S[16, 15]
- ***ic_val_data***: entrada binaria que informa de que existe una muestra válida en *id_data*
- ***clk***: entrada de reloj
- ***od_data***: salida del dato filtrado, posee formato S[14, 13]
- ***oc_val_data***: salida binaria que informa de que existe una muestra válida a la salida del filtro

Las especificaciones del filtro son:

- Arquitectura paralela, con implementación cableada de los multiplicadores
- Número de coeficientes: $N=3$
- Los coeficientes son simétricos $[-1/16, 1-1/8, -1/16]$
- El modelo del filtro deberá ser sintetizable y podrá ser implementado en un dispositivo FPGA Cyclone IV EP4CE115F29C7, funcionando a una frecuencia de reloj de 125 MHz.
- La frecuencia máxima que deberá ser filtrada es de 40 MHz

3.1. Ficheros

Los ficheros necesarios para realizar esta sección son:

- `conf_comp_dac.m`: script de Matlab que presenta los coeficientes del filtro y grafica la respuesta en frecuencia del DAC y del filtro compensador
- `comp_dac.slx`: modelo Simulink del filtro compensador del DAC
- `sim_comp_dac_model.m`: script de Matlab que configura el modelo Simulink y crea todos los archivos necesarios para el testeo del filtro
- `*comp_dac_pc.sv`: modelo system verilog del filtro compensador del DAC con precisión completa
- `comp_dac.sv`: modelo system verilog del filtro compensador del DAC con precisión recortada a un formato S[14, 13]
- `*comp_dac_pkg.sv`: paquete para configurar el banco de pruebas de los módulos `comp_dac_pc` y `comp_dac`. Se generará desde Matlab
- `*comp_dac_tsb.sv`: banco de pruebas de los módulos `comp_dac_pc` y `comp_dac`
- `comp_dac_wrp.sv`: fichero "wrap" del módulo `comp_dac` para la medida de la frecuencia máxima de funcionamiento
- `*comp_dac_tsb.tcl`: Fichero para ejecutar la simulación en modo comandos, desde la consola de Modelsim
- `*comp_dac_tsb.do`: Configuración de las formas de onda del banco de pruebas del módulo `comp_dac` en el visor de "waves" de Modelsim
- `comp_dac.sdc`: Fichero que contiene las restricciones temporales, para realizar el estudio de la velocidad del circuito.

NOTA: Los ficheros marcados con un "*" no están disponibles en Poliformat y los tendrá que generar el alumno.

3.2. Tareas a realizar

1. Dibuje el esquema circuital del filtro paralelo a implementar, teniendo en cuenta que los coeficientes son simétricos y los multiplicadores se implementan cableados.
2. Implemente en Simulink el filtro anterior y compárelo con el filtro ideal proporcionado en el fichero Simulink `comp_dac.slx`.
3. Estudie el crecimiento de la ruta de datos del filtro implementado y complete el script de Matlab para generar los ficheros de test y el paquete (`id_comp_dac.txt`, `od_comp_dac.txt` y `comp_dac_pkg.sv`).

4. Codifique, en System Verilog, el modelo del filtro compensador de la respuesta del DAC con precisión completa `comp_dac_pc.sv`.
5. Testee el modelo anterior siguiendo la misma estrategia que en prácticas anteriores.
6. Utilizando el modelo Simulink y el script de Matlab, cuantifique la salida del filtro con formato S[14,13].
7. Partiendo del módulo `comp_dac_pc`, codifique, en System Verilog, el modelo del filtro compensador de la respuesta del convertidor digital analógico con precisión recortada a 14 bits `comp_dac.sv`.
8. Testee el modelo anterior siguiendo la misma estrategia que en prácticas anteriores.
9. Estime el número de recursos hardware (LE aritméticos y FFs) que se requieren para la implementación del módulo `comp_dac`. Implemente el módulo `comp_dac` en el dispositivo Cyclone IV EP4CE115F29C7 e indique los recursos hardware que se requiere para su implementación (incluya pantallazo de la ventana Recursos de Quartus). Razone si los recursos obtenidos son coherentes con su estimación.
10. Escriba el código System Verilog del fichero `comp_dac_wrp.sv`, con un "wrap" para el módulo `comp_dac`, en el que se registren sus entradas y salidas, y obtenga la frecuencia máxima de funcionamiento en el dispositivo indicado.